Claude Suggests, Uppaal Verifies: Repairing a Lane-Change Model

Author Names

Institution

Abstract. A modeller gave the AI assistant Claude, connected to the Uppaal editor through a tool interface, a timed-automata model of cars that change lane and acceleration on a multi-lane road, and asked it to fix the model because two cars could collide. The session had a clear division of labour. Claude *suggested and explained*: it diagnosed the error from a counterexample, proposed a repair based on an invariant on stopping points, formulated the queries to check it, and interpreted traces and statistics for the modeller. Uppaal *verified and reported*: by symbolic model checking it returned verdicts and counterexamples, and by statistical model checking it returned probability estimates with confidence intervals and witness runs. We summarise the case and reflect on what each party contributed, where the evidence came from, and where the integration showed friction.

1 Setting

The timed-automata [1] model `IntCars13` (2 cars, 1 lane), open in the Uppaal editor [2], follows the multi-lane traffic setting of [4]. Time advances in steps of one time unit; a `Decisions` automaton sends `go[i]!` to each car, which then selects `a` $\in \{-1, 0, 1\}$ and a lane, and stores `acc = 2*a`. After each step all cars move at once:

```
pos[i] = pos[i] + vel[i] + acc[i]/2;   vel[i] = vel[i] + acc[i];
```

A `Monitor` enters `ERROR` when both cars are on the same lane at the same position. Cars may move "providing that no collision will occur in the next time unit", using `brk_dist(v,a) = v*v/(2*a)`. The modeller's request was a single sentence: *"From the queries it follows that the two cars can collide. Can you fix the model."*

Division of labour. Through the tool interface Claude could read and edit declarations, edges and queries, and ask Uppaal to check syntax, verify queries and return traces and plots; it never saw the GUI. Table 1 shows who did what. Every verdict, counterexample and estimate in this summary was reported by Uppaal; every diagnosis, model change, query and explanation was suggested by Claude; and the modeller decided, intervening four times: to ask about lane changes, to redirect the analysis to statistical model checking (SMC), to stop a long-running engine, and to ask how a trace had been obtained.

Table 1. Roles in the session.

Claude *suggests and explains*	UPPAAL *verifies and reports*	Modeller *decides*
diagnoses errors	from syntax check of every edit	states the problem
traces		
proposes model fixes (guard, monitor)	symbolic: verdicts (sat./not sat.),	accepts or redirects
argues why a fix is correct	shortest counterexamples, witnesses	(e.g. "use only SMC")
formulates queries	statistical: estimates with confidence	owns the property
explains traces and statistics	intervals, witness runs, plots	(e.g. the finish line)
reduces instances checking	for time and memory per query	stops runs

2 Claude Diagnoses and Repairs

UPPAAL reports, Claude diagnoses. For the modeller's query `E<> Monitor.ERROR` UPPAAL reported *satisfied* after 0.05 s together with a shortest counterexample of 21 transitions. Claude read the trace and explained it: neither a lane change nor a moving leader is involved; car 1 at position 7 with speed 2 is 3 behind a standing car 2; the guard compares the gap with `brk_dist` of the *current* speed ($3 > 1$), allows acceleration, and one step later both cars are at position 10. The check ignores the distance driven during the step, the larger braking distance after accelerating, and what the other car does in the same step.

Claude's argument for the fix. The following reasoning is Claude's; UPPAAL did not check it. Velocities start at 0 and change by ± 2, so they are always even, and braking from $v = 2k$ covers $(2k-1) + (2k-3) + \cdots + 1 = v^2/4$. Under the integer update `brk_dist(v,2)` is therefore the *exact* stopping distance. With $stop(p, v) = p + v^2/4$ the repaired model maintains, for every follower f behind a leader ℓ on the same lane,

$$I(f, \ell): \quad p_f < p_\ell \ \wedge \ stop(p_f, v_f) < stop(p_\ell, v_\ell).$$

Two observations make I inductive: no admissible action decreases a car's stopping point (full braking leaves it unchanged), and if I holds then braking keeps it whatever the leader does. Hence a safe action always exists, which yields collision freedom and absence of deadlock from one argument, independently of the order in which the cars decide. This is a discrete analogue of the safe distance of responsibility-sensitive safety [5].

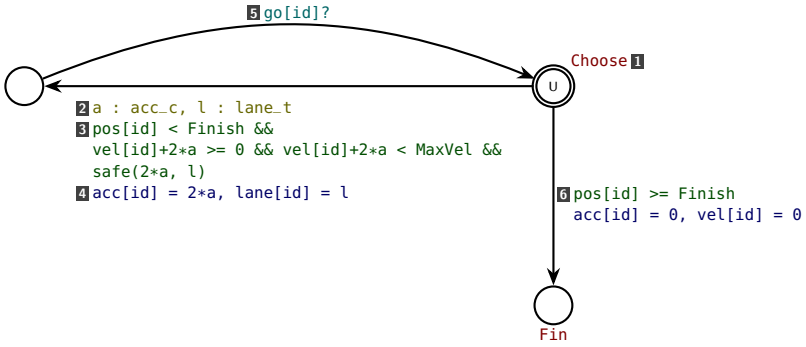
Claude's suggested guard. Claude replaced the two `overlap` functions by a single predicate `safe` in the local declarations of the `Car` template; Fig. 1 shows the complete repaired template with Claude's comments.

```
bool onroad(car_t c)      { return pos[c] < Finish; }        //
    finished cars have left
int  stop_pt(int p, int v) { return p + brk_dist(v,2); }     //
    exact stopping point
int  min_next_pos(car_t c) { return vel[c] ⩾ 2 ? pos[c]+vel[c]−1 :
    pos[c]; }
int  max_acc(car_t c)      { return vel[c]+2 < MaxVel ? 2 : 0; }

bool safe(int a2, lane_t l) {
  int p = pos[id] + vel[id] + a2/2;   int v = vel[id] + a2;
  for (o : car_t) if (o ≠ id && onroad(o) && lane[o] == l) {
    if (pos[o] > pos[id]) {          // o in front: assume o brakes
    fully
      if (!(p < min_next_pos(o) && stop_pt(p,v) <
    stop_pt(pos[o],vel[o])))
        return false;
    } else if (pos[o] < pos[id]) {   // o behind: only checked on a
    lane change,
      if (l ≠ lane[id]) {            // assuming o accelerates
    maximally
        int ao = max_acc(o);  int po = pos[o]+vel[o]+ao/2;  int vo
    = vel[o]+ao;
        if (!(po < p && stop_pt(po,vo) < stop_pt(p,v))) return
    false;
      }
    } else return false;             // same position
  }
  return true;
}
```

UPPAAL's syntax check rejected Claude's first version, which contained XML entities (`&&`) instead of plain text; the corrected one passed.

A second counterexample, and a fix that touched the property. For the repaired model on the shortened road of Sect. 3 UPPAAL again reported *satisfied*, now with a 65-transition counterexample. Claude explained it as a collision *behind* the finish line: a car crossing `Finish` sets its speed to 0, an infinitely hard brake that violates the leader assumption of I. Claude suggested treating finished cars as having left the road, ignored them in `safe`, and changed the `Monitor` guard to `pos[1]==pos[2] && lane[1]==lane[2] && pos[1]<Finish`. It stated this choice and named the alternative (cars braking to a stop after the finish), but the choice weakens the safety property and belongs to the modeller.



1 `Choose` is initial (double circle) and urgent (U): a car decides without letting time pass. Being initial, it also lets cars decide at time 0 before any `go` is sent, so car 1 may decide twice in the first instant.
2 Non-deterministic choice of acceleration $a \in \{-1, 0, 1\}$ (stored as ± 2) and of a target lane l; staying in the lane is $l =$ `lane[id]`. In SMC the choice is resolved at random among the enabled combinations.
3 Speed stays in $[0, \texttt{MaxVel})$; `safe` enforces the stopping-point invariant I against every car on lane l. Full braking (or standing still) always satisfies it, so a car on the road is never blocked: no deadlock.
4 Only the decision is recorded; positions and speeds change for all cars at once in `updateDiscrete()` of `Decisions`, one time unit later.
5 Broadcast from `Decisions` at each integer time point; the unnamed waiting location has no invariant, the car simply waits for its turn.
6 A car past `Finish` stops instantly and is regarded as having left the road: `onroad` excludes it from `safe`, and the `Monitor` ignores positions $\geq$ `Finish`.

Fig. 1. The repaired `Car(id)` template with comments. Labels use the colours of the Uppaal editor: select, guard, synchronisation and update. The only change to the original template is `safe(2*a, l)` in 3, which replaces the two `overlap` conditions.

3 Uppaal Verifies and Reports

Table 2 collects what Uppaal reported. For the original model a counterexample came back at once; a verdict that the repaired model is safe did not. At `Finish` = 700 Uppaal gave no answer: the exhaustive query outlived the 300 s tool call by several minutes, during which the engine answered every request with "busy". Claude therefore suggested two reductions: a shortened road, and removing the global clock `time`, which is never reset and only used by an SMC query. On that instance Uppaal reported the finish-line counterexample of Sect. 2 after 28 s and, after the repair, *not satisfied* for `E<> Monitor.ERROR` after 35 s. Whether the clock or the still-loaded engine caused the earlier time-out was not isolated. That neither reduction affects safety is again Claude's argument, not a Uppaal result.

Two lanes. With one lane the backward branch of `safe` is never exercised. When the modeller asked whether cars can still change lane, Claude set `LANES = 2` and formulated reachability queries; Uppaal returned witnesses for overtaking and merging back within 0.1 s. The exhaustive safety query, however, timed out after 400 s, with the engine grown to 5.5 GB. The modeller asked to stop Uppaal

Table 2. Verdicts and estimates reported by UPPAAL. Queries marked † were formulated by Claude, the others were in the modeller's file. Symbolic queries were run on a shortened road (`Finish` = 120) chosen by Claude; SMC queries use 1000 runs of 100 time units with 95 % confidence intervals.

Query	lanes	`Finish`	original	repaired
Symbolic model checking: verdict, counterexample or witness				
`E<> Monitor.ERROR`	1	700	sat., trace	no answer
`E<> Monitor.ERROR`	1	120	–	**not sat.** (35 s, 1.1 GB)
`A[]` no overtaking within a lane†	1	120	–	**sat.** (40 s)
`E<>` overtake, merge back†	2	120	–	sat., witness
Statistical model checking: estimate, witness run				
`Pr[<=100;1000](<> Monitor.ERROR)`	1	700	0.430 ± 0.031	$0/1000$, ≤ 0.004
`Pr[<=100;1000](<> Monitor.ERROR)`	2	120	0.040 ± 0.013	$0/1000$, ≤ 0.004
`Pr[<=100;1000](<> Monitor.ERROR)`	2	700	–	$0/1000$, ≤ 0.004
`Pr[...](<> lane[1]!=lane[2])`†	2	700	–	≥ 0.996
`Pr[...](<>` overtake, merge back`)`†	2	700	–	0.644 ± 0.030

and to use only SMC. Claude terminated the engine process, not the editor holding the unsaved model, and proposed the statistical queries in Table 2. UPPAAL reported lane changes in all 1000 runs, overtaking with merging back in about 64 %, and no collision. Claude's reading: the repair keeps the intended manoeuvres possible, and with 95 % confidence the collision probability within 100 time units is below 0.4 %: evidence, not a proof.

4 Claude Explains a Run Reported by UPPAAL

The modeller asked for an explanation of collisions in the *original* model with two lanes. Claude temporarily reinstated the original guard and `Monitor` and asked UPPAAL for the SMC estimate. UPPAAL reported 0.040 ± 0.013 (39 of 1000 runs collide) and, because the query carries the option `-diagnostic`, also returned one colliding run. Claude condensed that run into the table below and explained it:

time	car 1 (lane, pos, vel)	car 2 (lane, pos, vel)	decisions
1	0, 1, 2	0, 10, 0	car 2 moves to lane 1, car 1 accelerates
2	0, 4, 4	1, 10, 0	**car 2 cuts back into lane 0**, 6 ahead
3	0, 8, 4	0, 10, 0	gap 2 < braking distance 4: car 1 brakes
4	0, 11, 2	0, 11, 2	`ERROR`

The cut-in at time 2 passes `overlap_backward` because it compares the *current* gap 6 with car 1's current braking distance 4, ignoring that car 1 is at 8 one step later; from then on braking is the only admissible action, and it is too late. Claude drew two further observations from the trace: had car 2 stood

still, car 1 would have *passed through* it (8 to 11) without the equality-based `Monitor` noticing; and because the urgent initial location `Choose` lets cars decide before any `go` is sent, car 1 decides twice at time 0. By Claude's calculation the repaired guard rejects the cut-in: with car 1 accelerating maximally its stopping point after the step is $9 + 36/4 = 18 \not< 10$. For the repaired model in the same setting UPPAAL reported no collision in 1000 runs.

The modeller then asked how a colliding trace had been extracted from random runs. Claude's answer separated the sources of the evidence: the one-lane counterexample came from UPPAAL's symbolic search (a shortest witness), the two-lane trace from a single SMC run that UPPAAL selected and that Claude therefore cannot claim to be typical.

5 Reflections

Claude suggests, UPPAAL verifies. The split in Table 1 held throughout: Claude never asserted safety on its own authority, and UPPAAL never proposed a change. UPPAAL found the bug and confirmed the fix; Claude turned counterexamples into a diagnosis and the diagnosis into a repair. The repair rested on two facts no query returned: the exactness of `brk_dist` for even speeds and the monotonicity of stopping points. They turned a heuristic check into an inductive invariant, so UPPAAL's task became confirmation rather than search. But the same argument also justified checking only a reduced instance, and there Claude's reasoning stands in for a tool result. Mechanising such arguments, e.g. by an inductive-invariant check or a parametric proof, would move them to the "verifies" column; this is the integration of formal methods that the case calls for.

Two kinds of report, two kinds of explanation. UPPAAL's symbolic reports were verdicts with short, explanatory counterexamples, but exhaustive safety did not scale beyond one lane and a short road. Its statistical reports were estimates with confidence intervals on the full instance, plus witness runs that Claude could explain like counterexamples. The modeller's instruction to rely on SMC was a sensible division of labour, provided each statement says whether it rests on a verdict, an estimate or Claude's argument.

Repairs can change properties, and properties encode instance assumptions. Claude's finish-line choice altered the `Monitor`, and the no-overtaking property that UPPAAL verified for one lane is refuted for two lanes by UPPAAL's own witness for overtaking and merging back. UPPAAL verifies the property it is given; whether it is the right one is for the modeller to decide, and Claude should present such changes as decisions rather than as part of a fix. A second session on the same model produced a different guard, so the repair is a design choice too.

The interface shapes the work. Synchronous calls with time-outs suit second-scale queries, not exhaustive checks that block the engine for minutes; cancellation, per-query resource limits and visibility of the editor state would have helped. Open points are more cars and lanes, a `Monitor` that detects passing between samples, a comparison of guards by SMC [3], and a mechanised proof

of the invariant. This summary was written by Claude from the session record; all verdicts, traces and statistics in it are UPPAAL's reports.

References

1. Alur, R., Dill, D.L.: A theory of timed automata. Theor. Comput. Sci. **126**(2), 183–235 (1994)
2. Behrmann, G., David, A., Larsen, K.G.: A tutorial on UPPAAL. In: SFM-RT 2004. LNCS, vol. 3185, pp. 200–236. Springer (2004)
3. David, A., Larsen, K.G., Legay, A., Mikučionis, M., Poulsen, D.B.: UPPAAL SMC tutorial. Int. J. Softw. Tools Technol. Transf. **17**(4), 397–415 (2015)
4. Hilscher, M., Linker, S., Olderog, E.-R., Ravn, A.P.: An abstract model for proving safety of multi-lane traffic manoeuvres. In: ICFEM 2011. LNCS, vol. 6991, pp. 404–419. Springer (2011)
5. Shalev-Shwartz, S., Shammah, S., Shashua, A.: On a formal model of safe and scalable self-driving cars. arXiv:1708.06374 (2017)